

PubMed Backend Testing Complete Guide

Zero Se Samjhein Saare Test Files Ka Concept, Logic Aur Assertions

Language: Hinglish | Target: Backend Engineers & QA

Chapter 1: Backend Testing Ki Buniyad (Basics from Zero)

PyTest ek automated framework hai jo hamare Python backend (FastAPI) ke functions aur API endpoints ko test karta hai. Jab bhi hum command chalte hain `py -m pytest`, PyTest saari `test_*.py` files ko automatically dhoondta hai aur unke andar ke `test_*` functions ko execute karta hai.

Important Concepts:

- **Assert Statement:** *assert condition ek check hai. Agar condition True hai toh test PASS hota hai. Agar condition False ho jaye toh test FAIL ho jata hai.*
- **Unit Test:** *Kisi chote function ko akela (isolated) test karna.*
- **Integration / API Test:** *FastAPI server ke routes (e.g. `POST /api/search`) ko request bhej kar poora response JSON verify karna.*
- **E2E (End-to-End) Test:** *User ke actual workflow ko end-to-end simulate karna (e.g. typo query se lekar PubMed vector search tak).*

Overview of Test Files in Repository

File Name	Test Focus	Total Tests	Status
test_med_search_pipeline.py	med_search 15 core functions (embeddings, cosine, caching, evaluation)	15	PASSED
test_api.py	FastAPI endpoints (<code>/</code> , <code>/api/health</code> , <code>/api/evaluate</code> , <code>/api/search</code>)	4	PASSED
test_e2e_integration.py	Synonym recall, typo resilience, UX, cache latency, SLA p95	6	PASSED
test_mesh_guardrail.py	NLM MeSH taxonomy validation & hallucination rejection	3	PASSED
test_pipeline.py	Legacy & core pipeline, hybrid score formula, unclamped metrics	2	PASSED
test_resilience.py	HTTP client automatic retry on 503/429 & LRU cache bounds	3	PASSED
test_spell_check_mesh.py	MeSH fuzzy spell correction & speed SLA check	1	PASSED
test_env.py	Environment variables loading & configuration	1	PASSED

Chapter 2: File 1 — test_med_search_pipeline.py (Core Logic)

Yeh test file `med_search/app.py` se liye gaye 15 naye functions ko step-by-step test karti hai. Yeh dekhti hai ki query validation, LLM expansion, embeddings, reranking, filtering, display formatting aur citation verification bilkul sahi kaam kar rahe hain.

1. test_take_user_query_valid() & test_take_user_query_invalid()

Matlab: User jab query type karta hai toh validation kaise hoti hai.

Kaam: Valid query (e.g. ' heart attack treatment ') se extra spaces hatata hai. Agar query khaali ho ya len < 2 ho toh ValueError raise karta hai.

Assertion: take_user_query("") par pytest.raises(ValueError) verify hota hai.

2. test_clean_and_preprocess_query()

Matlab: Text cleaning aur sanitization check karna.

Kaam: Special characters (like ! ? %) hatata hai, lowercase karta hai, lekin medical hyphen (e.g. type-2) aur spaces retain karta hai.

Assertion: Output string 'heart attack risk factors 2026' ke barabar aati hai.

3. test_expand_query_with_llm_fallback()

Matlab: LLM API key na hone par fallback dictionary kaam karti hai ya nahi.

Kaam: Agar Gemini/Anthropic key empty ho, toh _FALLBACK_SYNONYMS se medical terms add karta hai.

Assertion: 'heart attack' query expand ho kar 'myocardial infarction' waale OR syntax me convert hoti hai.

4. test_build_pubmed_query_params()

Matlab: PubMed ESearch API parameter construction check karna.

Kaam: SearchFilters (journal, start_date, end_date) ko PubMed ke standard [Journal] aur mindate/maxdate parameters me transform karta hai.

Assertion: Output dict me params['db'] == 'pubmed' aur "'NEJM'[Journal]" maujood hota hai.

5. test_generate_embeddings_and_compute_similarity()

Matlab: Vector Embedding aur Cosine Matrix Similarity (@) test karna.

Kaam: SentenceTransformers('all-MiniLM-L6-v2') se texts ke 384-dimension vectors banata hai aur matrix dot product abstract_embeddings @ query_embedding se cosine score nikalta hai.

Assertion: Metformin query ke liye Metformin article ka score Hypertension article se zyada nikalta hai (scores[0] > scores[1]).

6. test_rerank_and_filter_results()

Matlab: High-to-low score sorting aur date range filter test karna.

Kaam: Articles ko similarity_score descending sort karta hai. apply_filters() out-of-bound years wale articles hata deta hai.

Assertion: Output me top article PMID '1' bachta hai aur filtered list me sirf valid year wala article aata hai.

7. test_format_display_and_citation_verification()

Matlab: Display formatting aur citation truth check karna.

Kaam: Abstract ko 280 characters par ... ke saath truncate karta hai, PubMed URL (<https://pubmed.ncbi.nlm.nih.gov/123/>) generate karta hai, aur verify_citation() se verify karta hai ki snippet source text se match karta hai.

Assertion: is_verified is True return hota hai.

8. test_evaluate_faithfulness_and_relevancy()

Matlab: RAGAS-style evaluation metrics (relevancy % & faithfulness ratio) calculate karna.

Kaam: Top results ka average similarity score (relevancy) aur snippet-in-abstract presence ratio (faithfulness) nikalta hai.

Assertion: metrics['answer_relevancy'] == 0.92 aur metrics['faithfulness'] == 1.0 pass hote hain.

Chapter 3: File 2 — test_api.py (FastAPI Endpoints)

Yeh file FastApi ke active routes ko TestClient(app) dwara HTTP requests bhej kar test karti hai. Yeh ensure karti hai ki frontend ko bilkul sahi JSON structure mil raha hai.

1. test_root_endpoint()

Endpoint: GET /

Kaam: Root welcome message, docs URL aur health check endpoint link verify karta hai.

Assertion: status_code == 200 aur data['health_check'] == '/api/health'.

2. test_health_check_endpoint()

Endpoint: GET /api/health

Kaam: System health, NLM MeSH descriptor load count (e.g. 30,000+ terms), LLM provider ('gemini'), aur embedding model status verify karta hai.

Assertion: data['status'] == 'healthy' aur data['ncbi_status'] == 'Operational'.

3. test_evaluate_endpoint_integrity_dynamic()

Endpoint: GET /api/evaluate?limit=1

Kaam: Live benchmark harness runs ko test karta hai. Test dataset ko mutate karke verify karta hai ki evaluation metrics dynamically change hote hain, yani static fake values nahi hain.

Assertion: Dataset expected PMID change karne par precision score p10 updated metric reflect karta hai.

4. test_search_endpoint_full_pipeline()

Endpoint: POST /api/search

Kaam: Full search payload (query, LLM expansion, MeSH guardrail, hybrid alpha) bhej kar poora pipeline trigger karta hai.

Assertion: Response me query, pubmed_query, concepts list, pipeline_logs (8 steps), execution_time_ms aur results list valid milti hain.

Chapter 4: File 3 — test_e2e_integration.py (End-to-End UX)

Yeh integration tests real-world medical search scenarios ko simulate karte hain.

1. test_e2e_synonym_recall()

Scenario: User ne 'heart attack' search kiya.

Expected Outcome: MeSH expansion ke wajah se output papers ke titles/abstracts me 'myocardial infarction' (clinical synonym) milna chahiye.

2. test_e2e_typo_resilience()

Scenario: User ne typos waali query likhi ('semaglutid cardiovascular outcoms').

Expected Outcome: System typo catch karta hai, corrected_query generate karta hai, aur clean query jitne hi quality papers fetch karta hai.

3. test_e2e_boolean_free_ux()

Scenario: User simple plain text likhta hai ('metformin therapy in type 2 diabetes') bina kisi complex Boolean tags ke.

Expected Outcome: Backend internally '("Metformin"[MeSH] OR metformin[tiab]) AND ...' Boolean structure taiyar karta hai.

4. test_e2e_cache_correctness_and_near_zero_latency()

Scenario: User same query do baar search karta hai.

Expected Outcome: Pehli baar API search hota hai aur result cache me save hota hai. Dusri baar call <100ms me cached: true ke saath return hota hai.

5. test_e2e_latency_sla_p95()

Scenario: Batch medical queries run karna (Metformin, Semaglutide, Aspirin, Statins, Pembrolizumab).

Expected Outcome: 95th percentile latency (p95 SLA) 2000ms (2 seconds) ke andar rehti hai.

6. test_e2e_precision_at_10_sanity()

Scenario: Benchmark harness endpoint (GET /api/evaluate) se dynamic metrics mangvana.

Expected Outcome: Precision@10, Recall@10, MRR, aur NDCG@10 float values 0.0 aur 1.0 ke beech valid multi hain.

Chapter 5: File 4 & 5 — test_mesh_guardrail.py & test_pipeline.py

File 4: test_mesh_guardrail.py (NLM Taxonomy Guardrail)

- 1. test_mesh_guardrail_accepts_valid_terms():** Real medical terms (Metformin, Asthma, Pembrolizumab, Melanoma) NLM MeSH database se match ho kar is_valid: True hote hain.
- 2. test_mesh_guardrail_rejects_hallucinations():** LLM dwara ghadhe gaye nakli/hallucinated terms (e.g. 'coronary pseudo-carditis', 'super-aspirin 9000') guardrail dwara strictly reject (is_valid: False) ho jate hain.
- 3. test_mesh_guardrail_disabled_passthrough():** Agar guardrail option OFF ho, toh terms status note me 'Guardrail disabled' ke saath passthrough ho jate hain.

File 5: test_pipeline.py (Core Math & Weighting Logic)

- 1. test_biomedical_spell_correction():** Single typo ('metformn' -> 'metformin') aur phrase typo ('mocardial infaction' -> 'myocardial infarction') fuzzy matching se fix hote hain.
- 2. test_extract_concepts():** Query se Drug ('metformin') aur Disease ('type 2 diabetes') categories extract karta hai.
- 3. test_build_pubmed_query():** Final PubMed query string me [MeSH] aur [tiab] tags match karta hai.
- 4. test_hybrid_score_weighting_formula_consistency():** Formula docstrings aur code consistency verify karta hai: **40% Semantic Vector + 25% BM25 Lexical + 15% MeSH + 10% Title + 10% Coverage.**
- 5. test_unclamped_metrics_on_garbage_query():** Garbage query dene par metrics me fake minimum floor (0.85) nahi lagti, balki honest 0.0 score milta hai.

Chapter 6: File 6, 7 & 8 — test_resilience.py, test_spell_check_mesh.py, test_env.py

File 6: test_resilience.py (Fault Tolerance & Retries)

- test_execute_with_retry_succeeds_on_first_try(): Normal HTTP request attempt 1 me hi return ho jata hai.
- test_execute_with_retry_recovers_after_transient_failure(): HTTP 503/429 network glitched requests par execute_with_retry automatic exponential backoff se retry karke 3rd attempt par recover ho jata hai.
- test_multi_tier_lru_cache(): Multi-tier LRU cache bounds aur term expansion keys verify karta hai.

File 7: test_spell_check_mesh.py & File 8: test_env.py

- test_spell_check_mesh.py: Offline MeSH fuzzy spell checker algorithm speed SLA check karta hai.
- test_env.py: Configuration file config.py me environment variables (NCBI_EMAIL, EMBEDDING_MODEL, CACHE_TTL_SECONDS) ki valid loading verify karta hai.

Chapter 7: Test Suite Ko Run Kaise Karein

Terminal par backend directory me jaa kar yeh command chalayein:

```
py -m pytest -p anyio -p no:httpbin
```

Result Expected: Saare 41 tests green pass hone chahiye! (41 passed in 40.5s).